

ARCQuant: Boosting NVFP4 Quantization with Augmented Residual Channels for LLMs

Haoqian Meng¹, Yilun Luo¹, Yafei Zhao¹, Wenyuan Liu¹, Peng Zhang¹¹, Xindian Ma¹

¹School of Computer Science and Technology, Tianjin University

Abstract

The emergence of fine-grained numerical formats like NVFP4 presents new opportunities for efficient Large Language Model (LLM) inference. However, it is difficult to adapt existing Post-Training Quantization (PTQ) strategies to these formats: rotation-based methods compromise fine-grained block isolation; smoothing techniques struggle with significant 4-bit quantization errors; and mixed-precision approaches often conflict with hardware constraints on unified-precision computation. To address these challenges, we propose ARCQuant, a framework that boosts NVFP4 performance via Augmented Residual Channels. Distinct from methods that compromise block isolation or hardware uniformity, ARCQuant maintains a strictly unified NVFP4 format by augmenting the activation matrix with quantized residual channels. This design integrates the error compensation process directly into the matrix reduction dimension, enabling the use of standard, highly optimized GEMM kernels with minimal overhead. Theoretical analysis confirms that the worst-case error bound of our dual-stage NVFP4 quantization is comparable to that of standard 8-bit formats such as MXFP8. Extensive experiments on LLaMA and Qwen models demonstrate that ARCQuant achieves state-of-the-art accuracy, comparable to full-precision baselines in perplexity and downstream tasks. Furthermore, deployment on RTX 5090 and RTX PRO 6000 GPUs confirms practical benefits, achieving up to 3× speedup over FP16. Our code is available at <https://github.com/actydef/ARCQuant>.

While 8-bit quantization and weight-only compression are widely adopted to reduce memory footprints (Dettmers et al., 2022; Lin et al., 2024), maximizing inference throughput requires quantizing both weights and activations to 4 bits (W4A4). However, it remains a formidable challenge to maintain model fidelity under such aggressive compression. Recently, hardware advancements such as the NVIDIA Blackwell architecture have introduced native support for fine-grained Microscaling formats, notably MXFP4 and NVFP4, as shown in Figure 1 (Nvidia, 2024; Darvish Rouhani et al., 2023; NVIDIA Corporation, 2024).

Microscaling formats like NVFP4 represent a significant advancement in efficient deep learning, demonstrating effectiveness in low-precision training and high-throughput inference (Chmiel et al., 2025; NVIDIA Corporation, 2024). Their primary advantage over coarse-grained quantization lies in fine-grained block-wise isolation, which prevents high-magnitude outliers from inflating scaling factors across entire tensors. However, it is insufficient to simply adopt this format for aggressive W4A4 constraints. Existing PTQ strategies often fail to exploit this structure, necessitating specialized algorithms that optimize quantization error while respecting NVFP4’s unique block-wise properties. Most W4A4 strategies designed for integer formats are ill-suited for the specific constraints of fine-grained NVFP4. Global transformations often prove counterproductive for block-wise scaling by redistributing outlier magnitudes across dimensions. This propagation inflates local dynamic ranges, thereby compromising the isolation capability inherent to the format. Furthermore, mixedprecision strategies such as Atom encounter significant hardware barriers.

1 Introduction

The deployment of Large Language Models (LLMs) faces severe constraints in memory bandwidth and computational latency. To address these bottlenecks, Post-Training Quantization (PTQ) has established itself as a standard optimization strategy (Xiao et al., 2024; Frantar et al., 2023).

¹ Corresponding author: pzhang@tju.edu.cn

The NVFP4 standard requires a block size of 16, creating a structural

comparable to that of standard single-stage MXFP8. This theoretically validates our

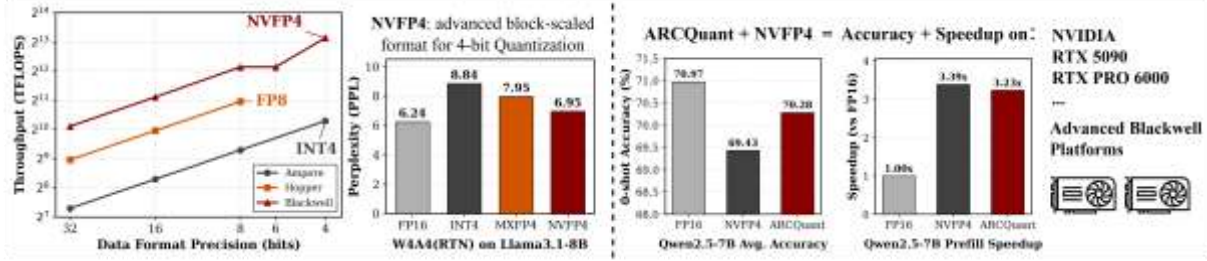


Figure 1: ARCQuant closes the NVFP4 accuracy gap while preserving high throughput on Blackwell platforms.

incompatibility with the coarser granularities of higher-precision formats. This discrepancy precludes the use of optimized Tensor Core instructions, which require unified data paths for highthroughput execution.

To overcome these algorithmic and hardware barriers, we propose ARCQuant (Augmented Residual Channels Quantization). Unlike mixed-precision approaches that violate hardware uniformity, ARCQuant maintains a strictly unified NVFP4 data path. Our method identifies dominant outlier channels and augments the input tensor with their quantized residuals. This facilitates a dual-stage quantization mechanism: the primary stage captures the highmagnitude structure, while the augmented stage recovers the fine-grained residual information. Crucially, this design maps the entire compensation process into the extended reduction dimension of a single matrix multiplication. Consequently, ARCQuant leverages standard and highly optimized GEMM kernels to achieve W4A8-level accuracy, while adhering to W4A4 hardware constraints.

Our contributions are summarized as follows:

- We propose ARCQuant, the first PTQ framework optimized for NVFP4 that utilizes augmented residual channels to bypass mixedprecision hardware constraints. Extensive experiments on LLaMA and Qwen families demonstrate that ARCQuant consistently outperforms existing NVFP4 adaptation strategies, delivering W4A8-level performance across various model families.
- We provide a rigorous analysis demonstrating that the worst-case error bound of our dualstage mechanism is

comparable to that of standard single-stage MXFP8. This theoretically validates our

- We implement high-performance kernels that map the complex residual accumulation process into standard, unified-precision GEMM calls, ensuring portability and efficiency. On RTX 5090 and PRO 6000, we achieve significant improvements in prefill speed and memory efficiency compared to FP16.

2 Related Work

Post-Training Quantization (PTQ) is pivotal for efficient LLMs. Early work identified activation outliers as the primary low-bit bottleneck (Dettmers et al., 2022). While initial methods focused on weight-only quantization (Lin et al., 2024), higher compression demands have shifted focus to activation quantization and fine-grained formats like NVFP4. We categorize existing strategies into mixed-precision, transformation-based, and compensation-based methods.

Mixed-precision Methods. These approaches mitigate quantization error by retaining sensitive information in higher precision. Early methods like Atom (Zhao et al., 2024) and QUIK (Ashkboos et al., 2023) adopt a selection-based strategy, keeping the majority of weights and activations in INT4 while preserving a subset of outlier channels in INT8 or FP16. Beyond simple channel selection, decomposition-based approaches such as ResQ (Saxena et al., 2025) and SVDQuant (Li et al., 2024) extract outliers into a separate high-precision low-rank branch, leaving the residual bulk in 4bit. Recent works have extended mixed-precision to emerging hardware formats. MicroMix (Liu et al.,

2025) explores mixing MXFP4 and MXFP8 on the NVIDIA Blackwell architecture. Similarly, FGMP (Hooper et al., 2025) proposes a strategy combining NVFP4 with FP8; while currently lacking native hardware support, it outlines a viable direction for future architectural designs.

Transformation-based Methods. These strategies reshape distributions to suppress outliers.

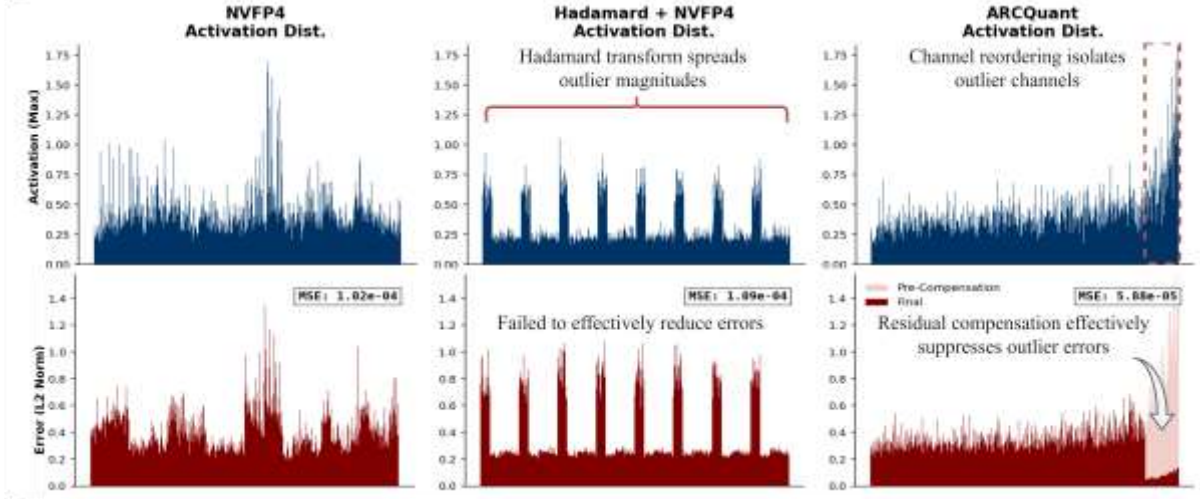


Figure 2: Activation magnitudes (blue) and quantization errors (red) on Llama 3.1-8B o_proj. ARCQuant isolates outlier to effectively suppress errors via residual compensation, whereas Hadamard spreads outlier magnitudes. SmoothQuant (Xiao et al., 2024) migrates quantization difficulty to weights, proving effective for INT8. For 4-bit scenarios, methods like QUIP (Tseng et al., 2024), QuaRot (Ashkboos et al., 2024), and FlatQuant (Sun et al., 2024) employ randomized rotations or learnable affine transformations to flatten activation distributions. However, the effectiveness of such global transformations on fine-grained formats is contested. Recent studies like BRQ (Shao et al., 2025) indicate that these operations may disrupt local block statistics—a critical limitation for NVFP4 that we further analyze in Section 3.

Compensation-based Methods. Strategies such as GPTQ (Frantar et al., 2023) and APTQ (Guan et al., 2024) minimize reconstruction error by utilizing Hessian-based optimization to adjust quantized weights. While highly effective for static parameter compression, these methods do not address the dynamic, runtime compensation of activation quantization errors that ARCQuant targets.

3 Methodology

3.1 Preliminary and Motivation

Problem Definition. We consider the linear layer computation $Y = XW^T$. Following the notation in QServe (Lin et al., 2025), we denote the configuration with x -bit weights and y -bit activations as $W \times A_y$. ARCQuant specifically targets

the W4A4 setting. Let g denote the quantization group size and $q_{\max}$ be the maximum representable value. For each group, the low-bit representation Q_X and the

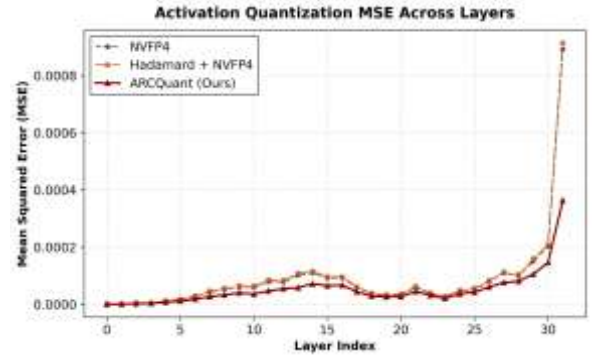


Figure 3: o_proj in Llama 3.1-8B, ARCQuant consistently suppressed MSE on NVFP4 across all layers.

scaling factor s_X are calculated as:

$$Q_X = \text{round}\left(\frac{X}{s_X}\right), \quad (1)$$

$$s_X = \frac{\max(|X|)}{q_{\max}}.$$

The dequantized approximation is given by $Q(X) = s_X \cdot Q_X$. Our objective is to minimize the reconstruction error $\|Y - Q(X)Q(W)^T\|_F$.

Block-scaled Quantization. NVIDIA Blackwell architecture introduces hardware acceleration for block-scaled formats. Unlike per-tensor methods, quantization is performed on small groups. Specifically, MXFP4 ($g = 32$) comprises 32 E2M1 elements sharing one E8M0 scaling factor, whereas NVFP4 ($g = 16$) uses 16 E2M1 elements with a shared E4M3 scale (details in Appendix A). Due to the limited dynamic range of E4M3, NVFP4

while preserving the original numerical values of the remaining elements.

Hardware Constraints on Mixed-Precision.

Conventional mixed-precision strategies like Atom protect sensitive channels using higher precision formats such as INT8 or FP16. However, this approach is inefficient for Blackwell NVFP4 acceleration. While NVFP4 operates with a group size of $g = 16$, standard higher-precision formats like MXFP6 and MXFP8 are designed for $g = 32$. Crucially, current Tensor Core instructions do not support efficient Matrix Multiply-Accumulate

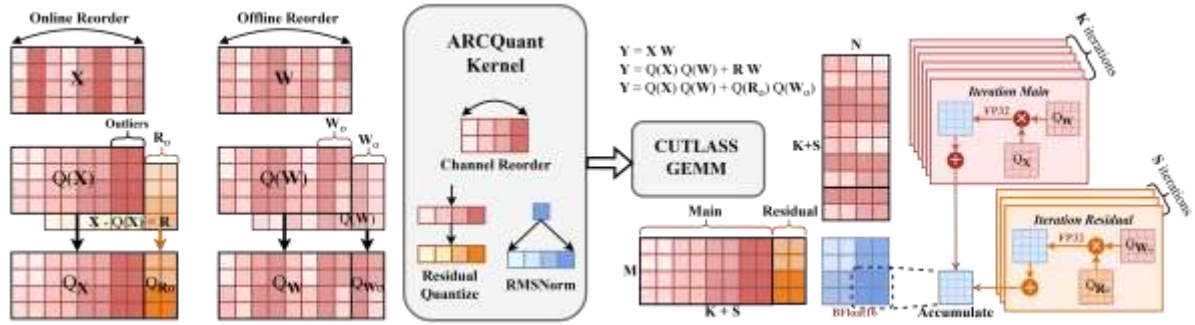


Figure 4: ARCQuant fuses main and residual computations by mapping them into the extended dimension.

requires a secondary per-tensor scaling factor. This fine-grained approach isolates outliers within specific blocks, preventing maximum values in outlier channels from inflating the scaling factors of adjacent low-magnitude elements. ARCQuant is specifically designed to leverage this isolation property to mitigate quantization errors arising from activation outlier channels.

Limitations of Global Rotations. Rotation strategies typically aim to suppress outliers by flattening the magnitude distribution across channels. While this reduces the global row-wise maximum, we observe it is detrimental for finegrained NVFP4. As shown in Figures 2 and 3, the Hadamard transform involves a linear combination that propagates the high magnitude of outlier channels into all dimensions. While the global peak decreases, this operation significantly increases the local dynamic range of previously low-magnitude blocks. This effect negates the isolation benefit of fine-grained scaling. In contrast, ARCQuant isolates outlier channels via reordering, performing targeted compensation

(MMA) operations on operands with heterogeneous group sizes. Mixing granularities precludes the use of these hardware-accelerated pipelines, necessitating complex kernel logic for memory alignment and significantly degrading inference throughput.

Motivation. These limitations present a critical dilemma: Global rotations propagate outlier magnitudes, while mixed-precision formats violate hardware uniformity. This raises the question: How can we mitigate outlier errors in fine-grained NVFP4 without these drawbacks? To this end, we propose *ARCQuant*. Instead of transforming input values or mixing data formats, ARCQuant augments the linear layer with residual channels of identical W4A4 precision. This design allows targeted error compensation via the standard NVFP4 GEMM kernels, effectively achieving high-fidelity reconstruction while strictly adhering to hardware constraints.

3.2 Augmented Residual Channels Strategy

Adaptive Outlier Identification. We aim to maximize reconstruction accuracy with minimal

cost by identifying channels that strictly require compensation. Utilizing calibration data, we pre-determine both the channel reordering indices and the number of outlier channels S . We first reorder channels based on their absolute maximums, adopting the sorting strategy from Atom (Zhao et al., 2024). We then determine the layer-wise maximum M , which defines the dynamic range of a per-tensor FP8 (E5M2) reference. We set the selection threshold $\tau = 2^{-3}M$. This reflects the 3-bit difference in exponent width between the reference E5M2 (5 bits) and the target E2M1 (2 bits). Values below this threshold ($|x| \leq \tau$) fall into the lower range of the FP8 format. In this range, the precision of NVFP4 is comparable to the baseline, making additional compensation unnecessary. Therefore, we focus solely on the top- S reordered channels exceeding this boundary, where the precision gap is significant due to the limited dynamic range. Detailed statistics on the layer-wise assignment of S on different models are shown in Figure 7.

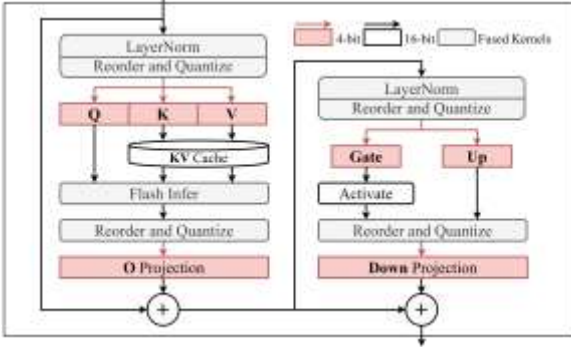


Figure 5: ARCQuant on a Transformers block in LLM.

Online Activation Quantization. We process X via: (1) *Reordering and Primary Quantization*. X is reordered, followed by block-wise quantization $Q_X = \text{round}(X/s_X)$. (2) *Residual Compensation*.

We isolate outliers X_o , compute residuals $R_o = X_o - s_{X_o} \cdot Q_{X_o}$, and quantize them into Q_{R_o} . (3) *Augmentation*. We concatenate along dimension K as $Q_{X_{\text{aug}}} = [Q_X \mid Q_{R_o}]$ with combined scales $s_{X_{\text{aug}}} = [s_X \mid s_{R_o}]$.

Offline Weight Quantization. Weights are aligned offline via: (1) *Reordering*. W is reordered to match X and quantized into Q_W . (2) *Augmentation*.

Instead of computing residuals, we duplicate the quantized outlier weights Q_{W_o} . We construct $Q_{W_{\text{aug}}} = [Q_W \mid Q_{W_o}]$, ensuring the GEMM effectively computes the correction term $R_o Q(W_o)^\top$.

Unified GEMM Execution. The original matrix multiplication (N, K_{in}, M) is transformed into an augmented operation $(N, K_{\text{in}} + S, M)$. The mathematical equivalence is derived as follows:

$$\begin{aligned} Y &\approx Q(X)Q(W)^\top + Q(R_o)Q(W_o)^\top \\ &= s_{X_{\text{aug}}} \cdot Q_{X_{\text{aug}}}(s_{W_{\text{aug}}} \cdot Q_{W_{\text{aug}}})^\top. \end{aligned} \quad (2)$$

By constructing $Q_{X_{\text{aug}}}$ and $Q_{W_{\text{aug}}}$ in the same precision, the compensation is seamlessly integrated into a single GEMM call.

3.3 ARCQuant Kernel Design

While compensation-based weight quantization incurs one-time offline costs, ARCQuant requires dynamic, online residual calculation. Achieving this efficiently poses significant challenges for CUDA kernel design. To address this, we implement a Fused Quantization Kernel that integrates Channel Reordering, RMSNorm, Primary Quantization, and Residual Quantization into a single operation. We utilize coalesced memory access patterns during reordering to minimize latency, and output data in the exact Interleaved Channel Layout (detailed in Appendix D) for the subsequent GEMM.

The output of this fused kernel is an augmented tensor strictly adhering to the NVFP4 format. As matrix multiplication dominates the computational load, this allows us to leverage standard, optimized CUTLASS GEMM kernels without modification, as shown in Figures 4 and 5. This design ensures hardware-native efficiency and simplifies deployment. Mechanically, the GEMM operates on the extended reduction dimension $(K_{\text{in}} + S)$. Due to the linearity of the accumulation, it seamlessly sums the primary computation (first K_{in} channels) and the residual corrections (final S channels) directly into the high-precision output accumulator.

3.4 Error Bound Analysis

Our analysis confirms that, for compensated outlier channels, the dual-stage mechanism

Table 1: Zero-shot, few-shot accuracy and perplexity. Bold indicates the best performance among W4A4 methods.

Method	Zero-shot Accuracy ($\uparrow$)						PPL ($\downarrow$)	MMLU ($\uparrow$)
	Arc-C	Hella	Lamba	PIQA	Wino	Average	WikiText2	5-shot
Llama 3.1-8B								
FP16								
W4A8 + RTN	50.51	77.62	73.16	79.82	71.82	70.59	7.07	61.08
FlatQuant	51.54	76.56	73.32	79.16	71.98	70.51	6.95	61.33
Atom	47.53	74.22	69.45	78.02	69.46	67.74	7.52	59.27
53.41		78.91	75.28	81.34	73.88	72.56	6.24	65.15
ARCQuant	52.22	77.22	72.79	79.16	73.09	70.90	6.87	62.61
Qwen2.5-7B								
FP16	51.28	78.98	71.63	79.71	73.24	70.97	6.85	74.16
W4A8 + RTN	51.19	77.30	67.22	79.71	70.32	69.15	7.44	71.88
FlatQuant	50.00	76.59	68.81	79.43	71.03	69.17	7.88	71.95
Atom	48.38	74.63	68.62	77.64	68.59	67.57	8.96	68.17
ARCQuant	51.79	77.88	70.68	79.05	71.98	70.28	7.28	72.84
Qwen2.5-32B								
FP16	55.89	84.17	76.17	82.43	75.45	74.82	5.02	83.26
W4A8 + RTN	57.94	83.10	74.79	82.43	76.72	75.00	5.41	82.57
FlatQuant	56.23	82.79	75.41	81.50	74.19	74.02	5.74	81.52
Atom	54.78	82.38	75.92	81.45	73.48	73.60	5.83	79.54
ARCQuant	56.57	83.36	76.05	82.64	75.37	74.80	5.38	82.61

matches the worst-case error bounds of standard MXFP8.

Preliminaries. Let M be the dynamic range. The worst-case quantization error is bounded by $|e| \leq s\epsilon = \alpha M\epsilon$, where $\alpha = s/M \geq 1$ is the scale alignment overhead and ϵ is the precision limit. Specifically, we consider NVFP4 (E2M1, $\epsilon_4 = 2^{-2}$, scale E4M3) and MXFP8 (E4M3, $\epsilon_8 = 2^{-4}$, scale E8M0). Since $\epsilon_4^2 = \epsilon_8$, a dual-stage NVFP4 mechanism theoretically bridges the precision gap, matching the resolution of standard MXFP8.

Derivation of Error Bounds. For MXFP8, the E8M0 scale (powers of 2) implies an alignment factor $\alpha_{mx} \in [1, 2)$. The worst-case bound is:

$$B_{mx} = \alpha_{mx} M \epsilon_8 < 2 M \epsilon_8. \quad (3)$$

For ARCQuant, we use a two-stage process on outliers: (1) quantize $\mathbf{x}$ with scale $s_1 = \alpha_1 M$. The residual is bounded by $\|\mathbf{r}\|_\infty \leq \alpha_1 M \epsilon_4$. (2) then quantize the residual with scale $s_2 = \alpha_2 \|\mathbf{r}\|_\infty$. Substituting $\epsilon_4^2 = \epsilon_8$, the bound becomes:

$$\begin{aligned} |e_{arc}| &\leq s_2 \epsilon_4 \leq (\alpha_2 \alpha_1 M \epsilon_4) \epsilon_4 \\ &= (\alpha_1 \alpha_2) M \epsilon_8 = B_{arc} \end{aligned} \quad (4)$$

Comparison. We assess theoretical parity by comparing alignment factors: MXFP8 uses exponent-only E8M0 scales ($\sup \alpha_{mx} = 2$), whereas NVFP4 employs mantissa-coded E4M3 scales (2^{-3} step size) yielding $\sup \alpha_1 \alpha_2 =$

$1.125^2 \approx 1.266$. Since $1.266 < 2$, ARCQuant achieves a worst-case error bound competitive with MXFP8, confirming that the dual-stage mechanism effectively restores high-fidelity

representation for sensitive channels within strict W4A4 constraints.

alongside a W4A8 baseline implemented with MXFP4 weights and MXFP8 activations. For advanced PTQ frameworks, we evaluate

Table 2: Zero-shot, few-shot accuracy and perplexity. Bold indicates the best performance among NVFP4 methods.

Method	Zero-shot Accuracy ($\uparrow$)						PPL ($\downarrow$)	MMLU ($\uparrow$)
	Arc-C	Hella	Lamba	PIQA	Wino	Average	WikiText2	5-shot
Llama 3.1-8B								
NVFP4 + RTN	50.26	77.56	73.84	78.67	71.90	70.45	6.95	61.64
NVFP4 + Smooth	50.34	77.51	73.32	79.22	71.59	70.40	6.92	61.76
NVFP4 + QuaRot	51.37	77.40	72.64	79.00	70.48	70.18	6.99	61.73
ARCQuant	52.22	77.22	72.79	79.16	73.09	70.90	6.87	62.61
Qwen2.5-7B								
NVFP4 + RTN	51.19	77.55	70.37	78.73	69.30	69.43	7.29	72.06
NVFP4 + Smooth	51.62	77.56	70.70	79.05	69.85	69.76	7.29	72.33
NVFP4 + QuaRot	50.00	77.52	70.56	79.22	70.01	69.46	7.30	72.33
ARCQuant	51.79	77.88	70.68	79.05	71.98	70.28	7.28	72.84

4 Experiments

4.1 Experimental Setup

Models and Datasets. We evaluate ARCQuant on a diverse set of models, including Llama 3.18B (Grattafiori et al., 2024) and the Qwen2.5 family (7B, 32B, Coder-7B-Instruct, Math-7B-Instruct) (Qwen et al., 2025). The metrics include:

(1) *PPL* on WikiText2 (Merity et al., 2016); (2) *Reasoning*: 5-shot MMLU (Hendrycks et al., 2021) and avg. 0-shot on ARC-C, HellaSwag, PIQA, Winogrande, Lambada (Clark et al., 2018; Zellers et al., 2019; Lourie et al., 2021; Sakaguchi et al., 2019; Paperno et al., 2016), using lm-eval (Gao et al., 2024).; and (3) *Domain Tasks*: HumanEval, MBPP, GSM8K, CMATH (Chen et al., 2021; Austin et al., 2021; Cobbe et al., 2021; Wei et al., 2023). (See Appendix B for details).

Baselines. To ensure a comprehensive and fair evaluation, we benchmark ARCQuant against the FP16 baseline and a diverse set of state-of-the-art quantization methods using their official configurations. We first establish performance lower bounds using Round-To-Nearest (RTN) applied across NVFP4, MXFP4, and INT4 formats,

FlatQuant (Sun et al., 2024) and Atom (Zhao et al., 2024) using their original configurations, as their complex learning strategies and customized CUDA kernels are structurally incompatible with NVFP4. Additionally, we adapt SmoothQuant (Xiao et al., 2024) and QuaRot (Ashkboos et al., 2024) to the NVFP4 format to assess their viability in fine-grained blockwise quantization. Detailed implementation settings for all baselines are provided in Appendix C.

Hardware. We conduct efficiency evaluations on NVIDIA RTX 5090 and RTX PRO 6000 GPUs, measuring both kernel-level latency and end-to-end inference throughput. Detailed statistics regarding Table 3: Code generation performance on Qwen2.5Coder-7B-Instruct. We report pass@1 accuracy on HumanEval, HumanEval+ (HE+), Mbpp, and Mbpp+.

Method	Qwen2.5-Coder-7B-Instruct			
	HE	HE+	Mbpp	Mbpp+
FP16	84.1	79.9	80.4	67.2
Atom	80.5	76.2	74.5	63.2
ARCQuant	86.0	79.3	79.9	68.3

post-quantization memory footprint and quantization latency are provided in Appendix B.

4.2 Main Results

Accuracy Comparison. Table 1 reports the performance across Llama 3.1 and Qwen2.5 families. ARCQuant consistently achieves the best results among W4A4 methods, significantly outperforming Atom and FlatQuant. Notably, on Llama 3.1-8B and Qwen2.5-7B, ARCQuant surpasses the

all metrics, effectively suppressing outlier errors in the activation matrices.

4.3 Efficiency and Speed

Efficiency Analysis. We analyze the computational overhead of ARCQuant through both kernel-level benchmarks and end-to-end latency profiling. Figure 8(a) shows that the GEMM latency exhibits a strictly linear correlation with the number of augmented channels S . Crucially, the inset highlights that within the typical operating range

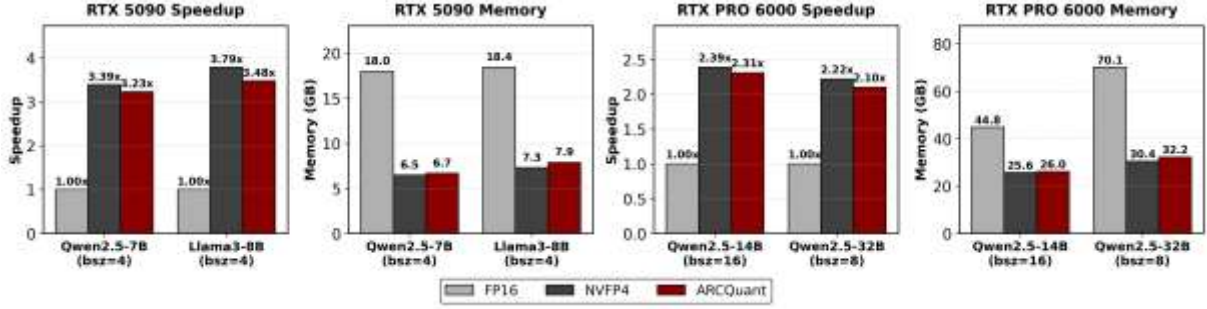
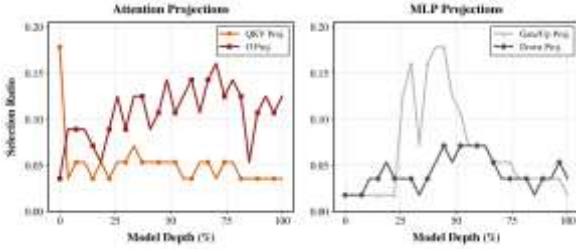


Figure 6: Prefill efficiency for various models with 2048 sequence length on RTX 5090 and PRO 6000.



W4A8 RTN baseline in both perplexity and MMLU scores, effectively closing the gap to full precision. For instance, on Qwen2.5-7B, our method reduces perplexity by 1.68 points compared to Atom. On the larger Qwen2.5-32B, ARCQuant demonstrates near-lossless compression, matching the FP16 baseline within a negligible margin.

Quantization Strategies on NVFP4. We evaluate four strategies: RTN, SmoothQuant, QuaRot, and ARCQuant. Table 2 shows that conventional PTQ methods yield diminishing returns on NVFP4. Notably, QuaRot (Hadamard transform) results in performance regression compared to RTN on Llama 3.1-8B, which confirms our motivation (Figure 2) that rotation disrupts the isolation benefits of fine-grained blocking. Similarly, SmoothQuant offers marginal gains due to the limited capacity of 4-bit weights. In contrast, ARCQuant consistently achieves the best results in

($S \leq 512$), ARCQuant incurs only marginal overhead over NVFP4 baseline, while maintaining a significant speed advantage over W4A8 and MXFP8 alternatives. This efficiency is further validated in the end-to-end breakdown on Qwen2.5-7B (Figure 8(b)), where ARCQuant results in a modest 4.9% total latency increase. Notably, the specific cost of our Fused Quantization Kernel remains minimal, confirming that our design effectively maps the complexity of residual compensation into efficient, unified-precision GEMM operations.

End-to-End Inference Performance. We evaluate prefill latency and memory on NVIDIA RTX PRO 6000 and RTX 5090. Detailed experimental results across varying sequence lengths are provided in Appendix B. Figure 6 confirms substantial gains over FP16: Qwen2.5-7B achieves

Figure 7: Outlier channels S across Qwen2.5-7B layers.

2.0 $\times$ –2.5 $\times$ speedup (PRO 6000) and Llama 3.18B reaches 3.5 $\times$ (RTX 5090), while memory usage drops by 1.5 $\times$ –2.8 $\times$. Crucially, residual overhead is marginal: compared to uncompensated NVFP4, latency increases by only 3%–9%.

4.4 Ablation Studies

Generalizability. Although tailored for the finegrained NVFP4 format, ARCQuant demonstrates robust extensibility to INT4 and MXFP4 formats. On Llama 3.1-8B, our method consistently outperforms RTN baselines across all metrics as detailed in Appendix B. Notably, ARCQuant reduces INT4 perplexity by 0.89 and improves zero-shot accuracy by 2.12%. These gains indicate that while the dualstage mechanism achieves optimal synergy with NVFP4, the underlying residual compensation principle remains effective in enhancing representation fidelity across various low-bit formats.

Calibration Robustness. We evaluate the sensitivity of ARCQuant to calibration data distributions in two scenarios. First, using standard text-domain calibration (WikiText2) for Qwen2.5-Coder-7BInstruct, we observe robust transfer to coding tasks (HumanEval, MBPP, and their extended versions), where ARCQuant retains over 99% of FP16 accuracy and significantly outperforms Atom (Table 3). Second, varying the calibration source (WikiText2, C4, HumanEval) (Merity et al., 2016; Raffel et al.,

2019; Chen et al., 2021) on Llama 3.1-8B results in marginal performance fluctuations (< 0.03 in PPL and $< 0.03\%$ in zero-shot accuracy) (see Appendix B). These results indicate that the outlier structures targeted by ARCQuant are stable, rendering the method robust to calibration data selection.

5 Conclusion

In this paper, we presented ARCQuant, a framework designed to bridge the gap between finegrained NVFP4 quantization and the outlier-heavy nature of LLM activations. Unlike rotation-based or mixed-precision approaches that compromise block isolation or hardware uniformity, ARCQuant resolves the outlier challenge through an augmented residual channel strategy. Our theoretical analysis confirms that this dual-stage mechanism achieves a worst-case error bound comparable to standard MXFP8. We translated this theoretical advantage into practical speedups via custom fused kernels, which enable dynamic residual compensation with minimal overhead. Crucially, ARCQuant secures these accuracy gains while maintaining strict compatibility with unified-precision hardware. As architectures evolve toward lower precisions, our principle of "trading minimal compute dimensions for higher fidelity" offers a scalable, hardware-friendly pathway for efficient LLM inference. Future work will extend this methodology to sub-4-bit formats and larger-scale architectures.

Limitations

While ARCQuant offers a hardware-efficient pathway for NVFP4 inference, we identify the following limitations to be addressed in future work:

Integration with Advanced Weight Quantization. Our current framework focuses primarily on resolving the activation outlier challenge, utilizing a standard Round-to-Nearest (RTN) strategy for weights. While effective, ARCQuant is theoretically compatible with advanced compensationbased weight quantization algorithms (e.g., GPTQ or AWQ). Integrating these methods could further enhance model fidelity,

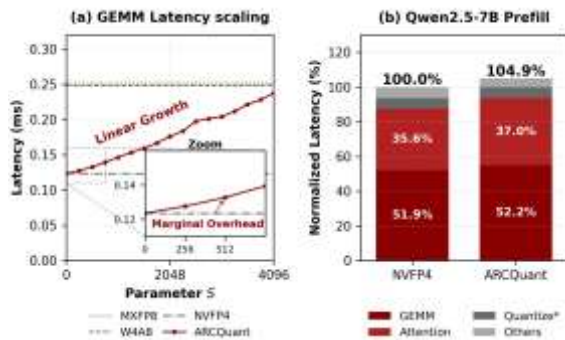


Figure 8: Efficiency Benchmarks. (a) Kernel latency vs. channel count S . The inset ($S \leq 512$) confirms ARCQuant is significantly faster than W4A8. (b) Qwen2.5-7B prefill breakdown showing 4.9% total overhead. *includes Reorder, RMSNorm, and Residual Quantize.

particularly for scenarios requiring sub-4-bit weight compression.

Hardware Dependency. ARCQuant is explicitly co-designed with the NVIDIA Blackwell architecture to leverage its unified-precision Tensor Core instructions. While our algorithmic contributions are general, the practical throughput benefits are contingent on hardware native support for the NVFP4 format. On legacy architectures lacking block-scaled acceleration, the method serves primarily as a simulation of future performance rather than an immediate deployment solution.

Dependence on Offline Calibration. The channel reordering indices and the outlier count S in ARCQuant are determined offline based on calibration data. Although our experiments demonstrate strong robustness across different calibration sets, this static approach inherently assumes that the distribution of outlier channels remains relatively consistent during inference. While this trade-off is necessary to avoid the high latency of runtime permutation search, it may theoretically limit adaptability to extreme out-of-distribution inputs compared to fully dynamic (albeit slower) strategies.

References

- Saleh Ashkboos, Ilia Markov, Elias Frantar, Tingxuan Zhong, Xincheng Wang, Jie Ren, Torsten Hoefler, and Dan Alistarh. 2023. Quik: Towards end-to-end 4-bit inference on generative large language models. *arXiv preprint arXiv:2310.09259*.
- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. *Quarot: Outlier-free 4-bit inference in rotated LLMs*. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. *Program synthesis with large language models*. *Preprint*, arXiv:2108.07732.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. *Evaluating large language models trained on code*. *arXiv preprint arXiv:2107.03374*.
- Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. 2025. Fp4 all the way: Fully quantized training of llms. *arXiv preprint arXiv:2505.19115*.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Bitar Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Venmugil Elango, Maximilian Golub, Alexander Heinecke, Phil JamesRoxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, and 14 others. 2023. *Microscaling data formats for deep learning*. *Preprint*, arXiv:2310.10537.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. *Llm.int8(): 8-bit matrix multiplication for transformers at scale*. *Preprint*, arXiv:2208.07339.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. *Gptq: Accurate post-training quantization for generative pre-trained transformers*. *Preprint*, arXiv:2210.17323.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, and 5 others. 2024. *The language model evaluation harness*.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad AlDahle, Aiesha Letman, and et al. Akhil Mathur. 2024. *The llama 3 herd of models*. *Preprint*, arXiv:2407.21783.
- Ziyi Guan, Hantao Huang, Yupeng Su, Hong Huang, Ngai Wong, and Hao Yu. 2024. Aptq: Attention-aware post-training mixed-precision quantization for large language models. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pages 1–6.

- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Coleman Hooper, Charbel Sakr, Ben Keller, Rangharajan Venkatesan, Kurt Keutzer, Sophia Shao, and Bruce Khailany. 2025. [Fgmp: Fine-grained mixed-precision weight and activation quantization for hardware-accelerated llm inference](#). *Preprint*, arXiv:2504.14152.
- Muyang Li, Yujun Lin, Zhekai Zhang, Tianle Cai, Xiuyu Li, Junxian Guo, Enze Xie, Chenlin Meng, Jun-Yan Zhu, and Song Han. 2024. Svdquant: Absorbing outliers by low-rank components for 4-bit diffusion models. *arXiv preprint arXiv:2411.05007*.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*.
- Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2025. [Qserve: W4a8kv4 quantization and system co-design for efficient llm serving](#). *Preprint*, arXiv:2405.04532.
- Wenyuan Liu, Haoqian Meng, Yilun Luo, Peng Zhang, and Xindian Ma. 2025. Micromix: Efficient mixed-precision quantization with microscaling formats for large language models. *arXiv preprint arXiv:2508.02343*.
- Nicholas Lourie, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. [Unicorn on rainbow: A universal commonsense reasoning model on a new multitask benchmark](#). *Preprint*, arXiv:2103.13009.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. [Pointer sentinel mixture models](#). *CoRR*, abs/1609.07843.
- Nvidia. 2024. [Nvidia blackwell architecture technical brief](#).
- NVIDIA Corporation. 2024. cuDNN Frontend API v1.14.0: Block-Scaling Operation. <https://docs.nvidia.com/deeplearning/cudnn/frontend/v1.14.0/operations/BlockScaling.html>. Accessed: 2025-09-16.
- Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernandez. 2016. [The LAMBADA dataset: Word prediction requiring a broad discourse context](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, Berlin, Germany. Association for Computational Linguistics.
- Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. [Exploring the limits of transfer learning with a unified text-to-text transformer](#). *CoRR*, abs/1910.10683.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2019. [Winogrande: An adversarial winograd schema challenge at scale](#). *Preprint*, arXiv:1907.10641.
- Utkarsh Saxena, Sayeh Sharify, Kaushik Roy, and Xin Wang. 2025. [Resq: Mixed-precision quantization of large language models with low-rank residuals](#). *Preprint*, arXiv:2412.14363.
- Yuantian Shao, Peisong Wang, Yuanteng Chen, Chang Xu, Zhihui Wei, and Jian Cheng. 2025. Block rotation is all you need for mxfp4 quantization. *arXiv preprint arXiv:2511.04214*.
- Yuxuan Sun, Ruikang Liu, Haoli Bai, Han Bao, Kang Zhao, Yuening Li, Jiaxin Hu, Xianzhi Yu, Lu Hou, Chun Yuan, and 1 others. 2024. Flatquant: Flatness matters for llm quantization. *arXiv preprint arXiv:2410.09426*.
- Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. [QuIP \$\\$\$: Even better LLM quantization with hadamard incoherence and lattice codebooks](#). In *Forty-first International Conference on Machine Learning*.
- Tianwen Wei, Jian Luan, Wei Liu, Shuang Dong, and Bin Wang. 2023. [Cmath: Can your language model pass chinese elementary school math test?](#) *Preprint*, arXiv:2306.16636.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2024. [Smoothquant: Accurate and efficient post-training quantization for large language models](#). *Preprint*, arXiv:2211.10438.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.
- Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. [Atom: Lowbit quantization for efficient and accurate llm serving](#).

In *Proceedings of Machine Learning and Systems*,
volume 6, pages 196–209.

A Block-scaled Numerical Formats

Recent hardware advances have introduced blockscaled data formats to bridge the gap between high compression rates and representational fidelity. Unlike traditional per-tensor quantization, these formats partition tensors into small, fixed-size blocks, where elements within each block share a common scaling factor. We summarize the key specifications of these formats in Table 7.

OCP Microscaling Formats (MXFP). The Open Compute Project (OCP) defines a family of Microscaling formats, including MXFP8, MXFP6, and MXFP4 (Darvish Rouhani et al., 2023). As shown in Table 7, these formats uniformly adopt a block size of $g = 32$. Crucially, they utilize an 8-bit scale factor in E8M0 format (exponent only), which acts as a shared exponent for the block. This design allows the elements (ranging from 4-bit to 8-bit) to share a dynamic range while maintaining efficient hardware implementation.

NVIDIA NVFP4. Distinct from the OCP standard, the NVFP4 format targets finer-grained quantization with a block size of $g = 16$ (NVIDIA Corporation, 2024). This reduced block size offers tighter isolation of outliers. However, unlike the exponent-only scales of MX formats, NVFP4 employs an E4M3 scale factor. Due to the limited dynamic range of E4M3 compared to E8M0, NVFP4 necessitates an additional, global FP32 tensor scale to align the block-wise values correctly. This hierarchical scaling structure (Element $\rightarrow$ Block Scale $\rightarrow$ Tensor Scale) is a unique characteristic of NVFP4 that requires specialized handling in software kernels.

B Experiment Details

B.1 Experiment Setup

Table 4: Quantization overhead and efficiency. We report the calibration latency, quantization time and model memory on RTX PRO 6000 GPU.

Models	Calib. (s) ↓	Quant. (s) ↓	Mem. (GB) ↓
Llama 3.1-8B	79.84	9.15	4.75
7B	89.66	9.38	4.24

Qwen2.5-32B 176.44 43.89 19.57

We used 128 samples from the WikiText2 (Merity et al., 2016) dataset, each with a sequence length

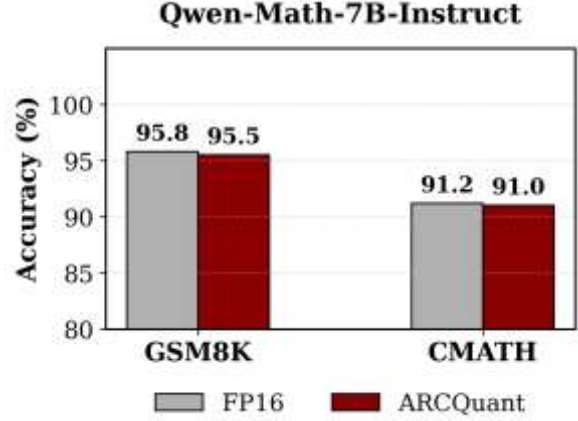


Figure 9: Qwen2.5-Math-7B-Instruct on GSM8K and CMATH

of 2048, as the calibration set. Based on the derived reordering indices and the layer-specific outlier counts S , we perform weight quantization. Table 4 reports the calibration latency, the resulting quantized model size, and the total quantization time.

B.2 Results Breakdown

Calibration Robustness. To assess the robustness of ARCQuant to calibration data variations, we conducted an ablation study using three distinct datasets: C4 (Raffel et al., 2019), WikiText2 (Merity et al., 2016), and HumanEval (Chen et al., 2021). To ensure a fair comparison, all calibration sets were controlled to have an equivalent sample size.

Table 5 details the resulting zero-shot accuracy and WikiText2 perplexity.

INT4 and MXFP4 Evaluation. To provide a comprehensive assessment of ARCQuant’s generalizability, we report the full zero-shot accuracy breakdown and perplexity scores on Llama 3.18B in Table 6. Detailed comparisons across five standard benchmarks (ARC-Challenge, HellaSwag, Lambada, PIQA, Winogrande) confirm that ARCQuant consistently outperforms the RTN baseline in both INT4 and MXFP4 formats.

Math Evaluation. We evaluate mathematical reasoning capabilities using Qwen2.5-Math-7BInstruct on the GSM8K and CMATH benchmarks. As shown in Figure 9, ARCQuant retains over 99% of the FP16 baseline accuracy, demonstrating exceptional robustness in domain-specific tasks.

End-to-End Inference. To supplement the

Code and Data. Our source code, compatible with PyTorch 2.9.0 and CUDA 12.8, is available at the anonymous GitHub repository: <https://github.com/actyepedef/ARCQuant>. We utilize publicly available datasets and official model checkpoints from Hugging Face without modification.

Implementation Details. For calibration, we

Table 5: Zero-shot accuracy and WikiText2 perplexity, with different calibration datasets(C4, HumanEval, WikiText2).

Calibration Set	Zero-shot Accuracy ($\uparrow$)						PPL ($\downarrow$)
	Arc-C	Hella	Lamba	PIQA	Wino	Average	WikiText2
Llama 3.1-8B							
C4	52.22	76.80	73.30	79.71	72.38	70.88	6.90
HumanEval	52.65	77.57	72.75	79.60	71.82	70.88	6.89
WikiText2	52.22	77.22	72.79	79.16	73.09	70.90	6.87

Table 6: Detailed zero-shot accuracy and perplexity on Llama 3.1-8B under INT4 and MXFP4 formats. Bold indicates the best performance within each quantization group.

Method	Zero-shot Accuracy ($\uparrow$)					Avg.	PPL ($\downarrow$)
	Arc-C	Hella	Lamba	PIQA	Wino	($\uparrow$)	WikiText2
FP16	53.58	78.90	75.47	81.18	74.03	72.63	6.24
INT4 Quantization							
RTN	45.99	73.82	66.14	77.37	65.59	65.78	8.84
ARCQuant	47.10	74.73	70.33	78.51	68.82	67.90	7.95
MXFP4 Quantization							
RTN	47.44	75.01	69.40	78.67	69.61	68.03	7.86
ARCQuant	48.46	75.51	72.13	78.18	70.56	68.97	7.50

efficiency analysis in Section 4, we provide the detailed breakdown of prefill performance. Table 8 reports the latency and peak memory consumption across varying batch sizes and sequence lengths

(512, 1024, 2048) on NVIDIA RTX PRO 6000 and RTX 5090 GPUs.

C Reproducibility Statement

We are committed to facilitating the reproduction of our results.

randomly select 128 segments from WikiText2 with a sequence length of 2048. The outlier threshold is set to $\tau = 2^{-3}M$. To guarantee deterministic behavior, we fix the random seed to 0 across all experiments.

Baselines and Hardware: Baselines (e.g., Atom, FlatQuant) are reproduced using their official repositories. All efficiency benchmarks reported in Section 4 were conducted on NVIDIA RTX 5090 and RTX PRO 6000 GPUs.

D Kernel Implementation Details

Interleaved Channel Layout. While our mathematical formulation in Section 3 denotes the augmented matrix as a simple concatenation ($[Q_X | Q_{Ro}]$), implementing this logically contiguous layout directly would incur significant latency due to strided global memory access patterns. To overcome this, we designed a hardware-friendly Interleaved Channel Layout for the actual kernel implementation. Specifically, for the S outlier

channels requiring compensation, we group them into blocks of 16 (matching the NVFP4 block size). Instead of separating all main quantizations and residual quantizations into distant memory regions, we interleave them locally: a 16-channel primary block is immediately followed by its corresponding 16-channel residual block in physical memory. This layout allows our Fused Quantization Kernel to compute both the primary and residual quantization values

Table 7: Comparison of parameters for MX-compliant formats and NVFP4. d and w denote the bit-width of elements and block scales, respectively. Notably, NVFP4 introduces an additional per-tensor scale in FP32.

Format Name	Element Bits (d)	Element Data Type	Bias (b)	Max Normal	Block Size (g)	Scale Type	Scale Bits (w)	Tensor Scale
MXFP8	8	FP8 (E5M2) FP8 (E4M3)	15 7	± 57344 ± 448	32	E8M0	8	N/A
MXFP6	6	FP6 (E3M2) FP6 (E2M3)	3 1	± 28 ± 7.5	32	E8M0	8	N/A
MXFP4	4	FP4 (E2M1)	1	± 6	32	E8M0	8	N/A
NVFP4	4	FP4 (E2M1)	1	± 6	16	E4M3	8	FP32

within on-chip registers and perform continuous, coalesced write-back operations to global memory. The weight matrix W is pre-processed offline to strictly match this interleaved pattern, ensuring that the standard GEMM operation yields the correct mathematical result without modification.

Compatibility and Future Work. As discussed in the Limitations, ARCQuant represents a quantization strategy for fine-grained formats rather than a hardware-specific workaround. As block-scaled formats like NVFP4 and MXFP4 gain wider adoption, the utility of our residual compensation mechanism becomes increasingly significant. A key advantage of our design is the decoupling of quantization logic from computation. Since the error compensation is mapped entirely into the input data space (via channel augmentation), ARCQuant is compatible with any standard, high-performance GEMM kernel (e.g., cuBLAS or CUTLASS) without requiring modifications to the inner loop of the matrix multiplication. Adapting ARCQuant to other emerging formats simply requires updating the Fused Quantization Kernel to support the

target format’s encoding, preserving the efficiency of the downstream GEMM. Finally, we plan to integrate ARCQuant into high-throughput serving engines such as vLLM to further validate its practical efficiency in production environments.

Table 8: Detailed prefill performance across various models and hardware configurations. We report the prefill latency (ms) and peak memory consumption (GB) for ARCQuant, FP16, and NVFP4. Bsz denotes batch size and Len denotes sequence length.

Setting (Bsz / Len)	Model	ARCQuant		FP16		NVFP4	
		Latency (ms)	Memory (GB)	Latency (ms)	Memory (GB)	Latency (ms)	Memory (GB)
Hardware: NVIDIA RTX PRO 6000							
32 / 512	Qwen2.5-7B	440.49	12.76	881.37	29.16	431.21	12.53
32 / 1024	Qwen2.5-7B	881.72	20.87	1895.40	44.08	857.81	20.64
32 / 2048	Qwen2.5-7B	1756.42	37.10	4162.91	73.92	1707.77	36.86
16 / 512	Qwen2.5-14B	393.66	17.73	885.61	36.22	383.10	17.28
16 / 1024	Qwen2.5-14B	793.21	26.05	1830.07	44.79	767.21	25.60
16 / 2048	Qwen2.5-14B	1592.07	42.67	4048.28	61.91	1544.15	42.22
8 / 512	Qwen2.5-32B	445.24	26.39	902.02	65.57	421.01	24.60
8 / 1024	Qwen2.5-32B	877.41	32.17	1844.20	70.11	830.58	30.38
8 / 2048	Qwen2.5-32B	1761.93	43.74	3908.75	79.18	1668.84	41.95
Hardware: NVIDIA RTX 5090							
4 / 512	Llama 3.1-8B	72.72	6.67	253.05	16.70	67.51	6.08
4 / 1024	Llama 3.1-8B	134.57	7.87	468.70	18.44	123.60	7.28
4 / 2048	Llama 3.1-8B	270.50	10.27	917.05	21.92	246.68	9.68
4 / 512	Qwen2.5-7B	69.27	5.65	226.83	16.10	66.95	5.44
4 / 1024	Qwen2.5-7B	128.17	6.67	414.51	17.97	122.20	6.45
4 / 2048	Qwen2.5-7B	251.33	8.70	888.14	21.70	237.60	8.48